

1. Introduction

The Supplier Management module is part of the Municipal Financial Management System (MFMS). Its purpose is to keep basic information about suppliers that provide goods or services to the municipality. The module allows the user to add suppliers, display registered suppliers and search for supplier information when it is needed.

I kept this module simple because Project A is the foundation version of the system. It uses the C programming concepts covered in the course, including arrays, strings, functions, loops and decision-making.

2. Supplier Information

The supplier record stores the information required in the project:

Field	Purpose
Supplier ID	A number used to identify the supplier.
Supplier Name	The name of the supplier or business.
Email	The supplier's email address.
Telephone Number	The supplier's contact number.
Town / Location	The town or location of the supplier.

3. Supplier Structure

I used a structure to keep all the information for one supplier together. An array of these structures can then be used to store several suppliers.

```
#define MAX_SUPPLIERS 100

typedef struct
{
    int supplierID;
    char name[100];
    char email[100];
    char telephone[30];
    char location[50];
} Supplier;

Supplier suppliers[MAX_SUPPLIERS];
int supplierCount = 0;
```

4. Add Supplier

The add supplier function asks the user for the supplier details and stores them in the array. It first checks that the supplier list is not full.

```
void addSupplier()
{
    if (supplierCount >= MAX_SUPPLIERS)
    {
        printf("Supplier list is full.\n");
        return;
    }

    suppliers[supplierCount].supplierID = supplierCount + 1;

    printf("Enter supplier name: ");
    fgets(suppliers[supplierCount].name,
        sizeof(suppliers[supplierCount].name), stdin);
    suppliers[supplierCount].name[
```

```

        strcpyn(suppliers[supplierCount].name, "\n")] = '\0';

printf("Enter email: ");
fgets(suppliers[supplierCount].email,
      sizeof(suppliers[supplierCount].email), stdin);
suppliers[supplierCount].email[
    strcpyn(suppliers[supplierCount].email, "\n")] = '\0';

printf("Enter telephone number: ");
fgets(suppliers[supplierCount].telephone,
      sizeof(suppliers[supplierCount].telephone), stdin);
suppliers[supplierCount].telephone[
    strcpyn(suppliers[supplierCount].telephone, "\n")] = '\0';

printf("Enter town/location: ");
fgets(suppliers[supplierCount].location,
      sizeof(suppliers[supplierCount].location), stdin);
suppliers[supplierCount].location[
    strcpyn(suppliers[supplierCount].location, "\n")] = '\0';

supplierCount++;
printf("Supplier added successfully.\n");
}

```

5. Display Suppliers

The display function uses a loop to go through the supplier array and show every registered supplier. If there are no suppliers, the user is informed.

```

void displaySuppliers()
{
    int i;

    if (supplierCount == 0)
    {
        printf("No suppliers have been registered.\n");
        return;
    }

    printf("\n--- Registered Suppliers ---\n");

    for (i = 0; i < supplierCount; i++)
    {
        printf("\nSupplier ID: %d\n", suppliers[i].supplierID);
        printf("Name: %s\n", suppliers[i].name);
        printf("Email: %s\n", suppliers[i].email);
        printf("Telephone: %s\n", suppliers[i].telephone);
        printf("Location: %s\n", suppliers[i].location);
    }
}

```

6. Search Supplier

The search function allows the user to search by supplier name. It checks the stored records and displays the supplier if a matching name is found.

```

void searchSupplier()
{
    char searchName[100];
    int i, found = 0;

    printf("Enter supplier name to search: ");
    fgets(searchName, sizeof(searchName), stdin);
    searchName[strlen(searchName)] = '\0';

    for (i = 0; i < supplierCount; i++)
    {
        if (strcmp(suppliers[i].name, searchName) == 0)
        {
            printf("\nSupplier found.\n");
            printf("ID: %d\n", suppliers[i].supplierID);
            printf("Name: %s\n", suppliers[i].name);
            printf("Email: %s\n", suppliers[i].email);
            printf("Telephone: %s\n", suppliers[i].telephone);
        }
    }
}

```

```

        printf("Location: %s\n", suppliers[i].location);
        found = 1;
        break;
    }
}

if (!found)
    printf("Supplier was not found.\n");
}

```

7. Supplier Menu

The supplier menu connects the functions and gives the user a simple way to choose an operation.

```

void supplierMenu()
{
    int choice;

    do
    {
        printf("\n===== SUPPLIER MANAGEMENT =====\n");
        printf("1. Add Supplier\n");
        printf("2. Display Suppliers\n");
        printf("3. Search Supplier\n");
        printf("4. Return to Main Menu\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();

        switch (choice)
        {
            case 1: addSupplier(); break;
            case 2: displaySuppliers(); break;
            case 3: searchSupplier(); break;
            case 4: break;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    } while (choice != 4);
}

```

8. Input Validation

Basic validation is used to prevent obvious errors. The program checks that the supplier array is not full and that invalid menu choices are handled. Supplier information is stored as strings because names, email addresses, telephone numbers and locations are text values. A telephone number is kept as a string because it can contain a leading zero or symbols such as +.

9. C Programming Concepts Used

Concept	Use in Supplier Management
Structure	Keeps all supplier details together.
Array	Stores multiple supplier records.
Strings	Stores names, email, telephone and location.
Functions	Separates adding, displaying and searching.
Loop	Moves through supplier records.
if / else	Handles validation and search results.
switch	Controls the supplier menu.
Input / Output	Allows the user to enter and view information.

10. Example Supplier Records

ID	Supplier Name	Email	Telephone	Location
1	Namibia Office Supplies	info@nos.com	061 234 567	Windhoek
2	Northern Equipment	sales@ne.com	065 222 111	Oshakati
3	Coastal Services	contact@cs.com	064 333 444	Swakopmund

These are example records that can be used when testing the supplier functions.

11. Testing the Supplier Module

Test	Expected Result
Add a valid supplier	Supplier is stored and a success message is displayed.
Display suppliers	All registered suppliers are displayed.
Search existing supplier	The matching supplier details are displayed.
Search unknown supplier	The system says the supplier was not found.
Invalid menu choice	The system displays an error and asks again.
Supplier list is full	The system prevents another supplier from being added.